🎯 USB Camera Intrinsic Calibration (Camera Calibration)

1. Course Content

Obtain the camera intrinsic parameters of the USB camera through checkerboard calibration board, used for publishing camera camera_info messages.

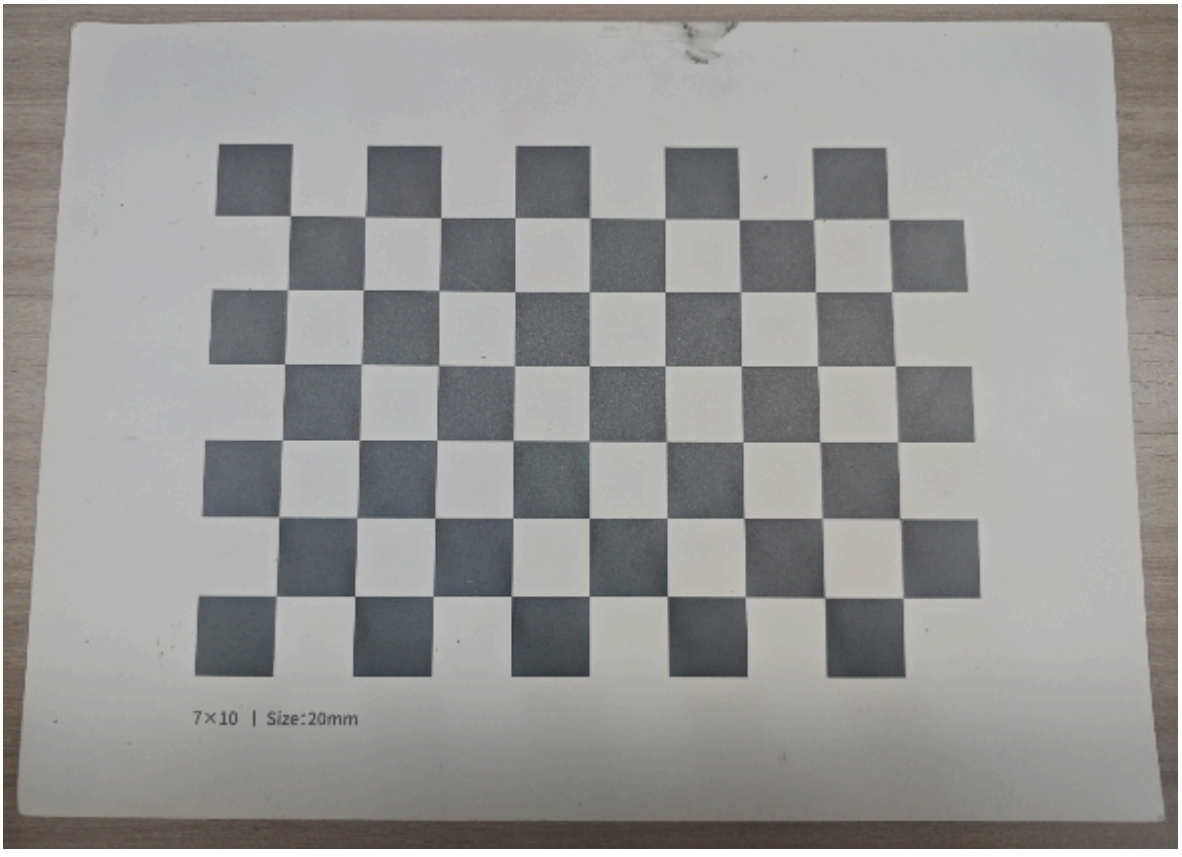
> **Important**
>
> - **Target Audience**: This course is for users with the **USB camera version**. Users with depth cameras can skip this step since the factory comes with chip-level calibration parameters.
> - **Factory Image Protection**: The factory system image already contains strictly calibrated camera intrinsic files. **Please do not arbitrarily overwrite factory parameters during first use!** This tutorial is for teaching calibration methods and recalibration when changing lenses.

---

2. Preparation

2.1 Prepare Checkerboard Calibration Board

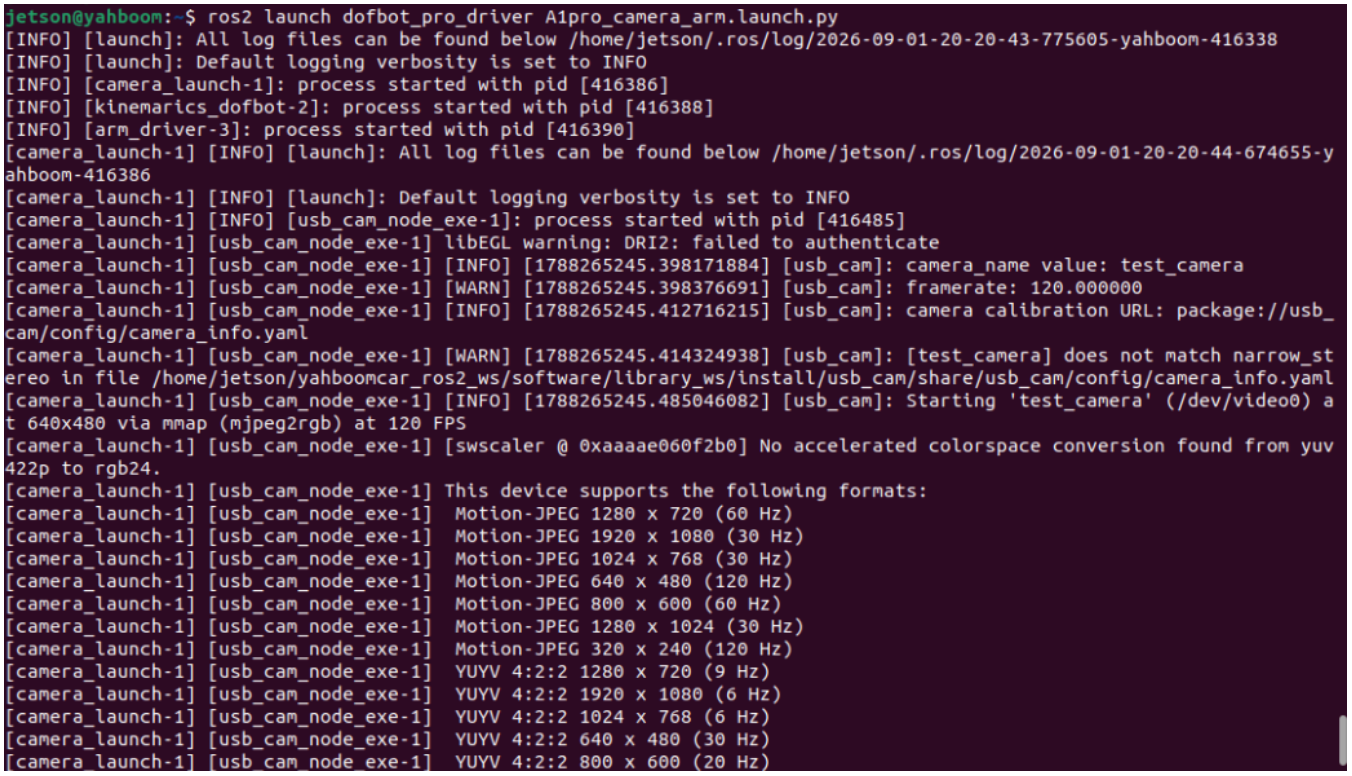
- Prepare a checkerboard calibration board, download and print the corresponding camera calibration board from the supporting materials [Attachments - Camera Checkerboard Calibration Board] and attach it to cardboard or other flat surface



2.2 Start Camera Driver and Calibration Program

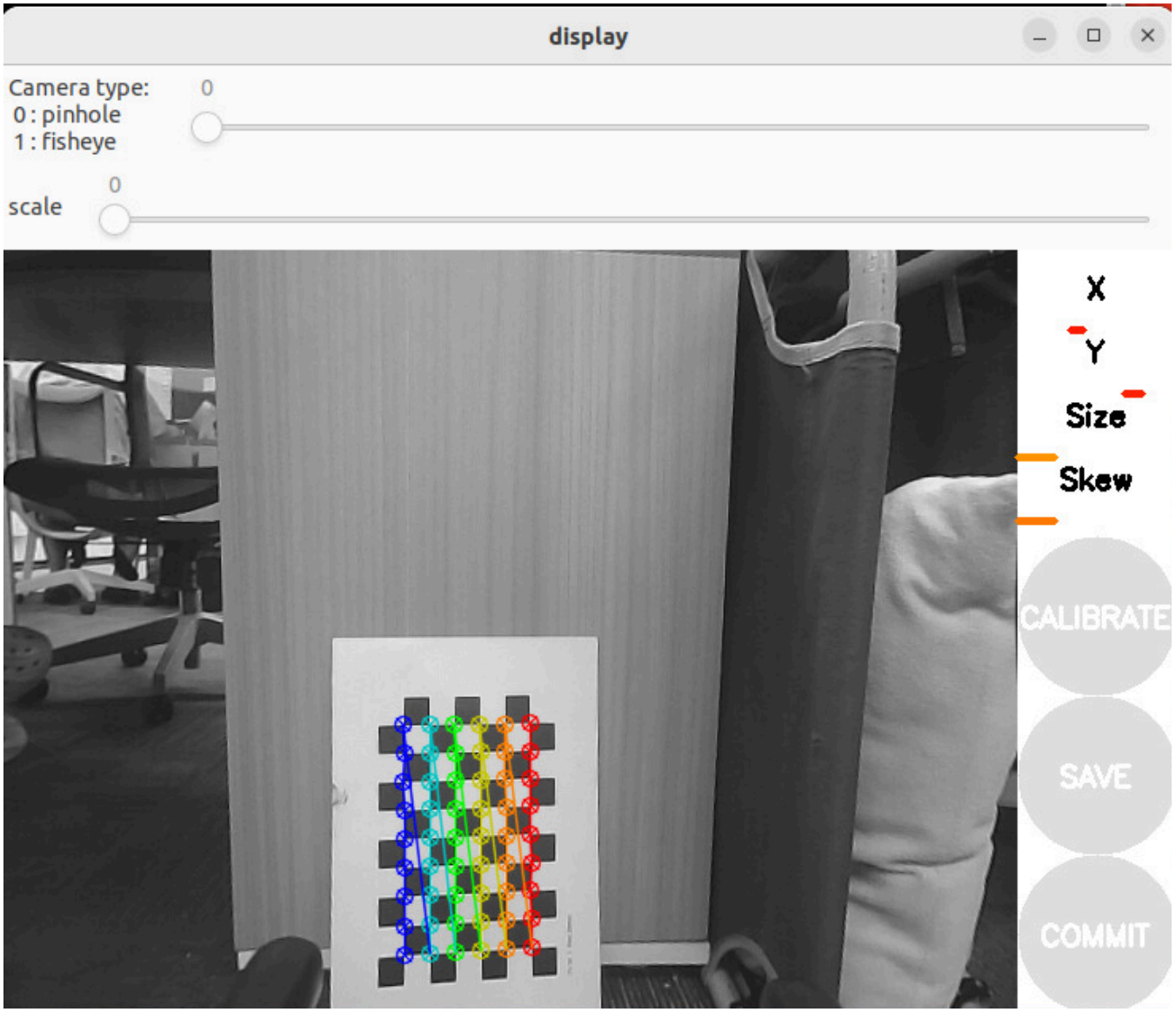
Open the terminal and run the camera intrinsic calibration launch file:

```
ros2 launch dofbot_pro_driver A1pro_camera_arm.launch.py
```



Run the calibration node (specify internal corner points and physical square size):

```
ros2 run camera_calibration cameracalibrator --size 9x6 --square 0.024 --ros-args -r image:=/usb_cam/image_raw
```

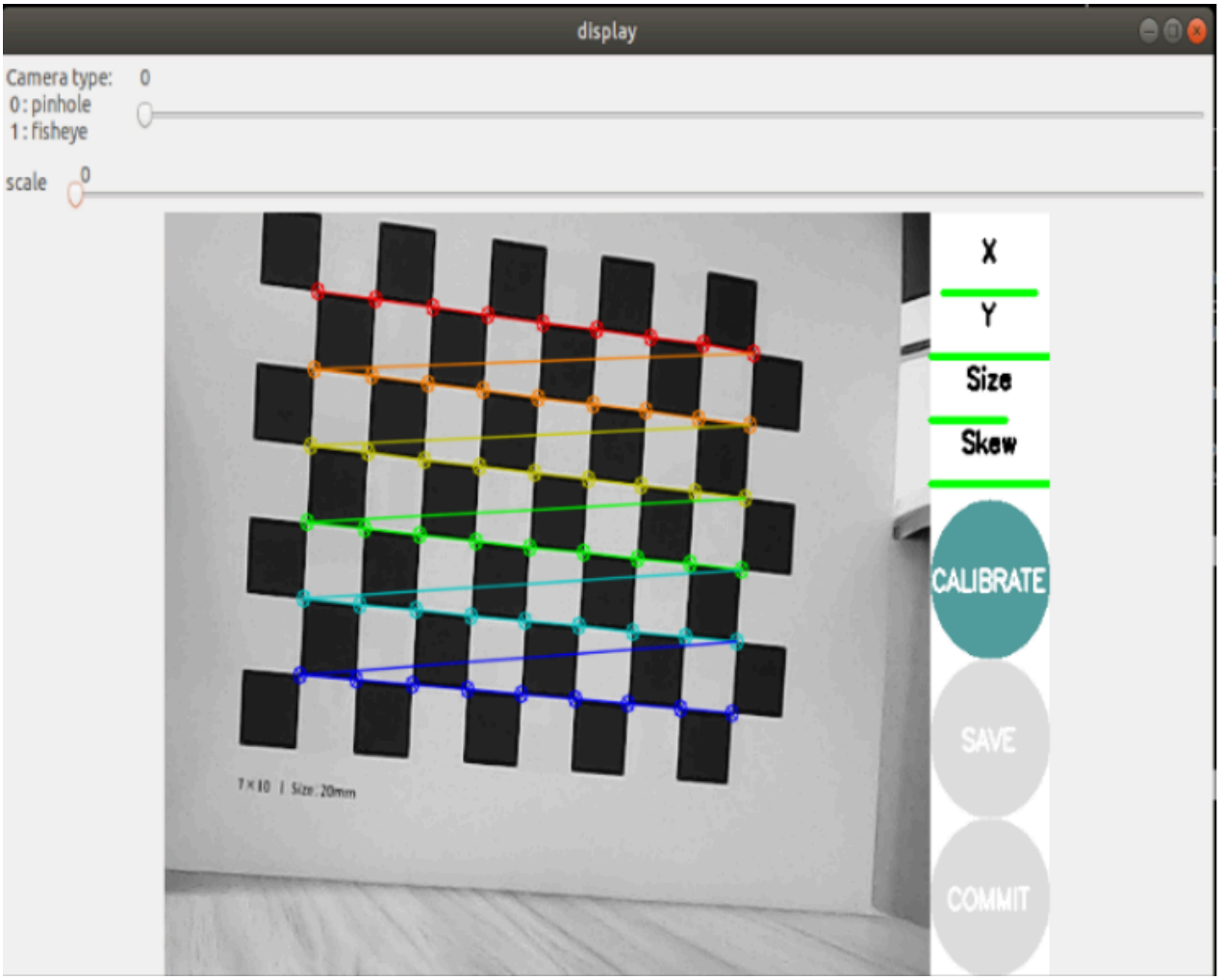


3. Calibration Operation Flow

3.1 Calibration Interface Description

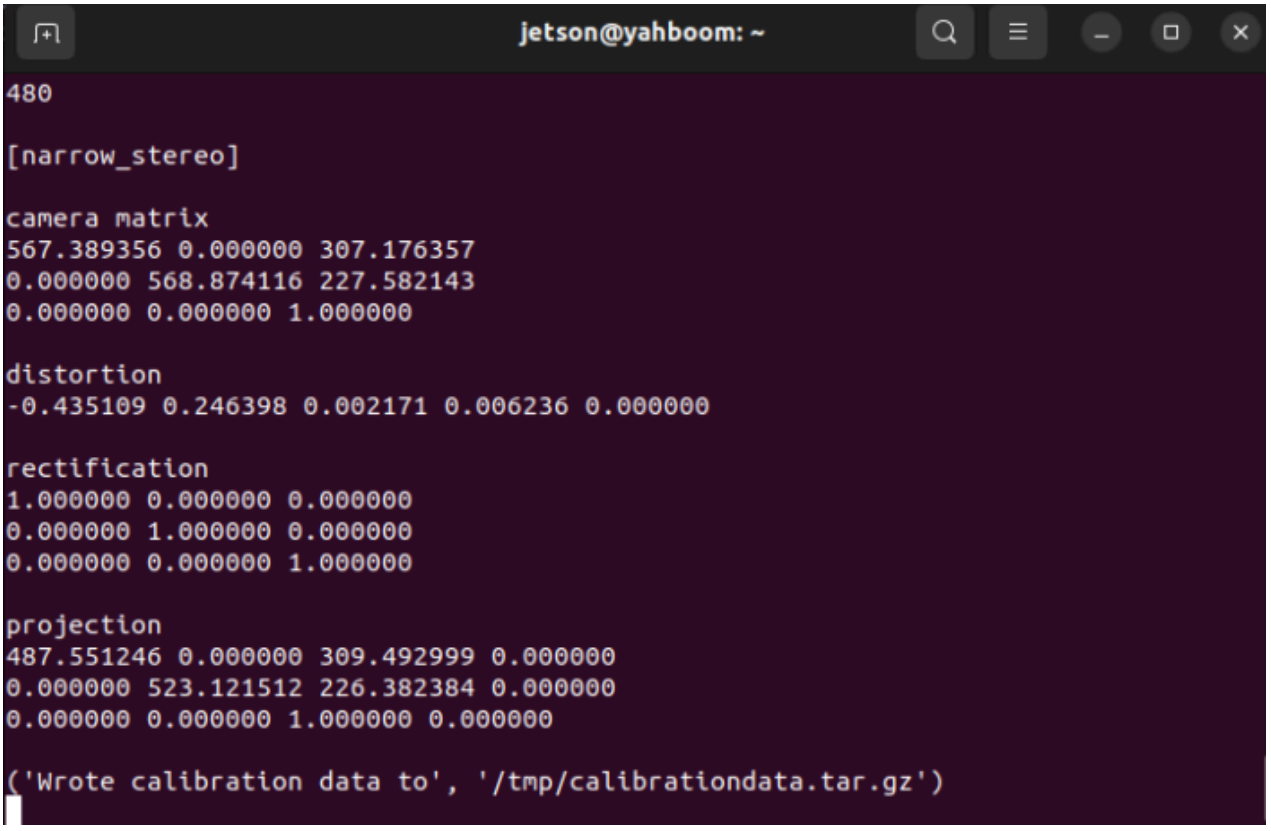
The right side of the calibration window shows four progress bars:

- `X`: Sample coverage for horizontal movement (left/right) of the checkerboard in the camera's field of view
- `Y`: Sample coverage for vertical movement (up/down) of the checkerboard in the camera's field of view
- `Size`: Coverage for depth distance as the checkerboard moves closer/farther from the camera
- `Skew`: Coverage for tilt, pitch, and roll rotation angles of the checkerboard in the field of view



3.2 Sampling and Calculation Steps

1. Hold the calibration board in front of the camera's field of view, slowly adjust the board position (up/down, left/right, closer/farther, and tilt/pitch/rotation);
2. When the algorithm successfully captures all internal corners, colored corner connection lines will be drawn in the image, and the corresponding progress bars on the right will change from red to yellow, and finally to green;
3. When all progress bars turn green and the `CALIBRATE` button in the lower right corner of the interface is lit and activated, click `CALIBRATE` to execute intrinsic parameter optimization (this process takes tens of seconds, please be patient);
4. After the solution is complete, the terminal will print the camera intrinsic matrix and distortion error. Click `SAVE` to save the calibration data to `/tmp/calibrationdata.tar.gz`;
5. Click `COMMIT` to submit and exit.

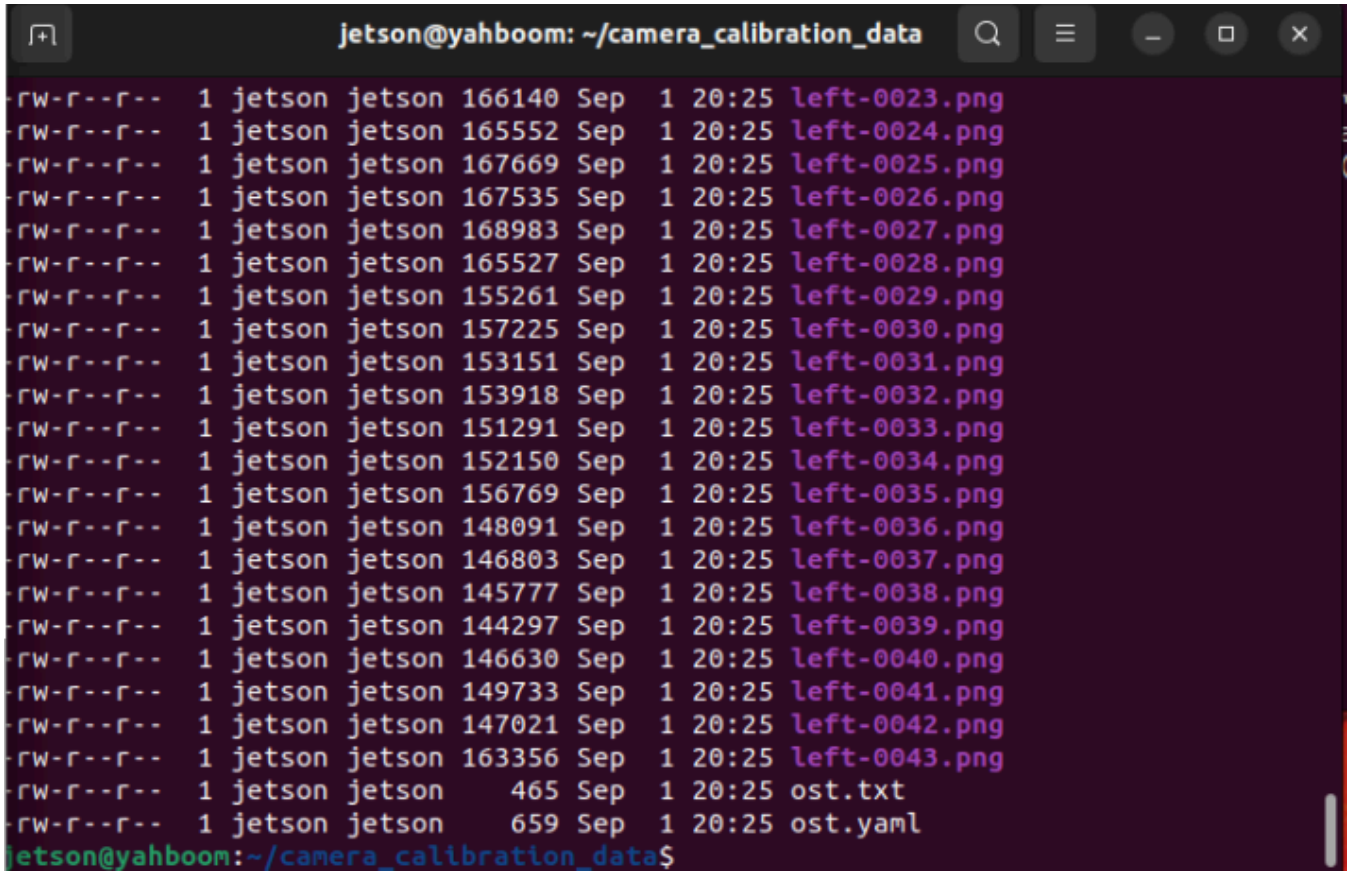


4. Export and View Calibration Results

4.1 Extract Calibration Package

Execute the extraction command in the terminal:

```
mkdir -p ~/camera_calibration_data
tar -zxvf /tmp/calibrationdata.tar.gz -C ~/camera_calibration_data/
cd ~/camera_calibration_data/
ls -la
```



After extraction, the directory contains all collected sample images, `ost.txt`, and the standard intrinsic file `ost.yaml`, which is the camera's intrinsic parameters.

4.2 Fill the Calibrated Intrinsics into These Two Files

- **Camera Startup Parameters**

  Startup command,

  ```
  cd ~/camera_calibration_data/
  cp ost.yaml
  ~/yahboomcar_ros2_ws/software/library_ws/src/usb_cam/config/camera_info.yaml

  # After replacement, compile
  cd ~/yahboomcar_ros2_ws/software/library_ws
  colcon build --packages-select usb_cam
  ```

- **Spatial 3D Grasping Camera Parameters**

  It is recommended to modify through vscode remote login, modify the following according to the intrinsic parameters from ost.yaml above:

  ```
  gedit
  ~/yahboomcar_ros2_ws/dofbot_pro_ws/src/dofbot_pro_usb/config/usb_camera_params.yaml
  ```

  Modify the parameters above



5. Camera Intrinsic YAML Structure Analysis

Standard Intrinsic File Reference:

```
~/yahboomcar_ros2_ws/software/library_ws/src/usb_cam/config/camera_info.yaml
```

```yaml
image_width: 640
image_height: 480
camera_name: narrow_stereo
camera_matrix:
  rows: 3
  cols: 3
  data: [567.38936,   0.     , 307.17636,
           0.     , 568.87412, 227.58214,
           0.     ,   0.     ,   1.     ]
distortion_model: plumb_bob
distortion_coefficients:
  rows: 1
  cols: 5
  data: [-0.435109, 0.246398, 0.002171, 0.006236, 0.000000]
rectification_matrix:
  rows: 3
  cols: 3
  data: [1., 0., 0.,
         0., 1., 0.,
         0., 0., 1.]
projection_matrix:
  rows: 3
  cols: 4
  data: [487.55125,   0.     , 309.493  ,   0.     ,
           0.     , 523.12151, 226.38238,   0.     ,
           0.     ,   0.     ,   1.     ,   0.     ]
```

- `image_width` / `image_height`: Camera image resolution, here 640×480 pixels
- `camera_name`: Camera name identifier
- `camera_matrix`: 3×3 camera intrinsic matrix K = [fx, 0, cx; 0, fy, cy; 0, 0, 1]
    - `fx`, `fy`: x/y direction focal length (pixel units), determines image scaling
    - `cx`, `cy`: Principal point coordinates (pixel position of optical center in image), ideally close to image center
- `distortion_model`: Distortion model, `plumb_bob` is a 5-parameter distortion model (k1, k2, p1, p2, k3)
- `distortion_coefficients`: Distortion coefficients
    - `k1`, `k2`, `k3`: Radial distortion coefficients, describing barrel/pincushion distortion caused by lens curvature
    - `p1`, `p2`: Tangential distortion coefficients, describing distortion caused by lens installation deviation
- `rectification_matrix`: 3×3 rectification matrix, for monocular camera it is identity matrix (no stereo rectification needed)
- `projection_matrix`: 3×4 projection matrix P = [fx', 0, cx', Tx; 0, fy', cy', Ty; 0, 0, 1, 0]
    - For monocular camera, Tx=0, Ty=0, used to project 3D points to 2D image plane